

PROJECT AND TEAM INFORMATION

Project Title

VidOptix — VIDEO COMPRESSION SYSTEM

Student / Team Information

Team Name: Team #	BitSquad DSCPP-III-2026-T198
Team member 1 (Team Lead)	Goswami, Anshika – 2510380062 2510380062@geu.ac.in 
Team member 2	Yadav ,Yaduveer– 2510370438 2510370438@geu.ac.in 
Team member 3	Chaudhary, Deepanshi – 2510370456 2510370456@geu.ac.in 
Team member 4	Ubaid,Mohd – 2510370084 2510370084@geu.ac.in 

PROPOSAL DESCRIPTION (10 pts)

Motivation (1 pt)

Problem Statement

Large videos require large data and storage space to save, share and transfer it. As the resolution and duration of video increase, the size of video increases. That's what compression is for.

Our project is to build a simple video compression system which will compress the video while keeping the quality as much as possible. The biggest thing we want to fix is the balance between the size of a video file and the quality of the video.

The system will allow us to lose some quality intentionally so as to make the file much smaller. To keep all the tiny details and to make a huge file. The project will also demonstrate how data structures and algorithms are used in life.

In particular it will use Huffman coding, priority queues or min-heaps and tree structures to tackle a real-world compression problem.

State of the Art / Current solution (1 pt)

Today video compression is mainly done using codecs like H.264/AVC, H.265/HEVC and AV1. These codecs use techniques to make videos smaller while still keeping the image sharp. Modern video codecs do more than just compress images; they also look at the similarities between images that follow each other. They use motion estimation and inter-frame prediction for this purpose.

Common methods used in today's video compression include

Transform coding: Changes image information into frequency sections to find details.

Quantization: Removes details to get a controlled level of quality loss.

Zigzag Scanning: Arranges the numbers in a way that helps with compression.

RLE: Makes repeated numbers, zeros smaller.

Huffman Coding: Makes the data smaller by using different lengths, for different parts.

Modern codecs do a job at compression but they are hard to create and have many parts that work together.

Project Goals and Milestones (2 pts)

Goals

1. To create a video compression system that makes videos smaller but keeps them looking good with about 75 to 80 percent of the original quality. 2. To Build a compression process that is easy to manage using DCT, quantization, zigzag scanning, RLE and Huffman coding.

1mm

3. To put the video compression techniques into a working system. The goal is to test and improve the system so it finds the balance between making the video smaller and keeping the video clear.

3mm

Milestones

1. Identifying the problem choosing the topic and doing research, on video compression.

1mm

2. Finalizing the compression process. Choosing OpenCV and FFmpeg for handling videos.

1mm

3.. Adding DCT, quantization, zigzag, RLE and Huffman coding into the system (to be completed by Week 1 to 5).

1mm

4.. Checking the system using compression ratio, file size, time to process and PSNR and SSIM (to be completed by Week 6 to 8).

1mm

5. Making improvements writing the instructions and showing the project (to be completed by Week 9 to 10).

Project Approach (3 pts)

The project will follow a frame-based video compression approach. The solution will be designed as a sequence of compression stages. Each stage will process the output of the previous stage.

The project will use a frame-based video compression method. The solution will be built as a series of compression stages. Each stage will take the output from the stage and process it further.

1. Video Frame Extraction:

The input video will be read using OpenCV or FFmpeg. These tools will handle video decoding, extracting frames and reading video data such as resolution and frame rate. The processing will happen one frame at a time to keep things and to use memory efficiently.

2. Transform and Lossy Compression:

Each frame will be split into 8×8 pixel blocks. Then the Discrete Cosine Transform or DCT will convert the pixel values into frequency coefficients. After that quantization will reduce the precision of the coefficients. The quantization level will be adjusted to control how compression we get versus how good the video looks.

3. Coefficient. Rle:

The quantized coefficients will be arranged using zigzag scanning. This method groups values and zeros together which helps in compressing the data better. Then Run-Length Encoding or RLE will be used to represent repeated values efficiently.

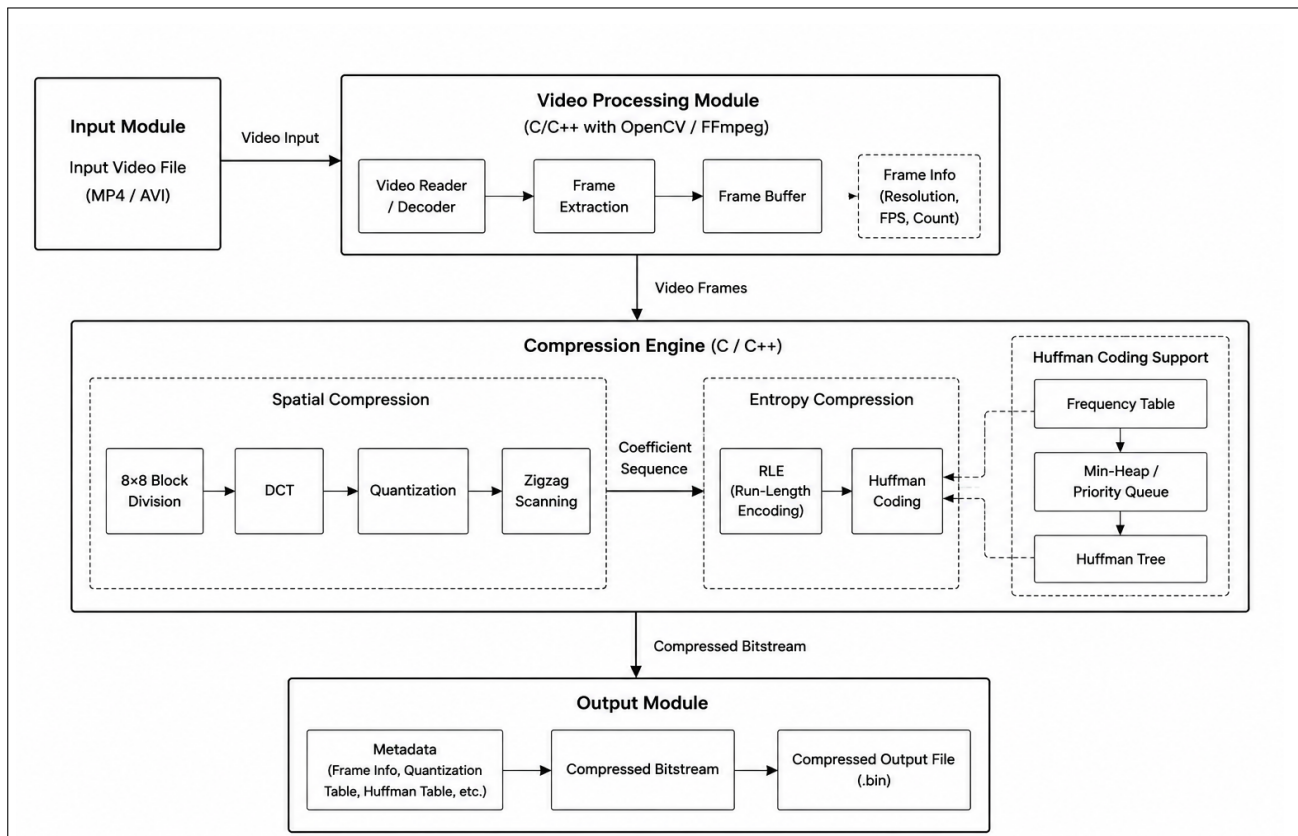
4. Huffman. Compressed Data:

The output from RLE will be further compressed using Huffman coding. A priority queue or min-heap will be used in C or C++ to build the Huffman tree. This tree will generate variable-length codes for the data. The final compressed binary data will be saved along with the metadata.

5.. Optimization:

Different quantization levels and sample videos will be tested. The goal is to find a balance between file size reduction and maintaining about 75 to 80 percent quality. The system will be tested using compression ratio, file size reduction, processing time and quality metrics such, as PSNR and SSIM.

System Architecture (High Level Diagram)(2 pts)



Project Outcome / Deliverables (1 pts)

The project will create a working video compression system based on frames. It will reduce the size of video files while keeping the quality at around 75 to 80 percent of the original. The goal is to make videos smaller without losing much clarity.ze while maintaining approximately 75–80 percent visual quality.

What we will deliver:

- 1.1. A working video compression pipeline that uses DCT, quantization, zigzag scanning, RLE and Huffman coding. Each step will be included to process the frames and shrink the file size.
2. A program that takes a video as input processes each frame and produces a compressed output file. The program will handle the compression workflow from start to finish.
3. A structured implementation of Huffman coding. This will use a priority queue or min-heap as the data structure to build the Huffman tree efficiently.
4. A quality and compression evaluation based on compression ratio, file-size reduction, processing time, and PSNR/SSIM.
5. A comparison of quantization levels. We will test how each level affects compression and quality to find the balance between file size and image appearance.
6. Full project documentation including source code, system design, architecture diagrams, test results and a final demonstration of the system in action.

The final output will show that combining transform-based methods like DCT with techniques, like Huffman coding and run-length encoding can significantly reduce video size while still maintaining a good level of visual quality.

Assumptions

Key assumptions include:

75-80 percent of visual quality is acceptable. Visual quality, at this level is considered fine.

Testing videos will have resolution and length so processing time remains manageable. Videos of size help keep the system quick.

OpenCV and FFmpeg will read videos. Extract frames. OpenCV and FFmpeg handle the reading part.

The proposed DCT, quantization, zigzag, RLE and Huffman steps will handle the compression process. DCT, quantization, zigzag, RLE and Huffman are the core stages.

References

1. OpenCV Documentation – Video I/O and Frame Processing:
<https://docs.opencv.org/4.13.0/dd/de7/group>
2. OpenCV – Reading and Writing Videos using OpenCV:
<https://opencv.org/reading-and-writing-videos-using-opencv/>
<https://opencv.org/reading-and-writing-videos-using-opencv/>
3. NIST Dictionary of Algorithms and Data Structures – Huffman Coding:
<https://xlinux.nist.gov/dads/HTML/huffmanCoding.html>
4. Wolfram MathWorld – Huffman Coding:
<https://mathworld.wolfram.com/HuffmanCoding.html>
5. RLE Discussion and Implementation:
<https://michaeldipperstein.github.io/rle.html>